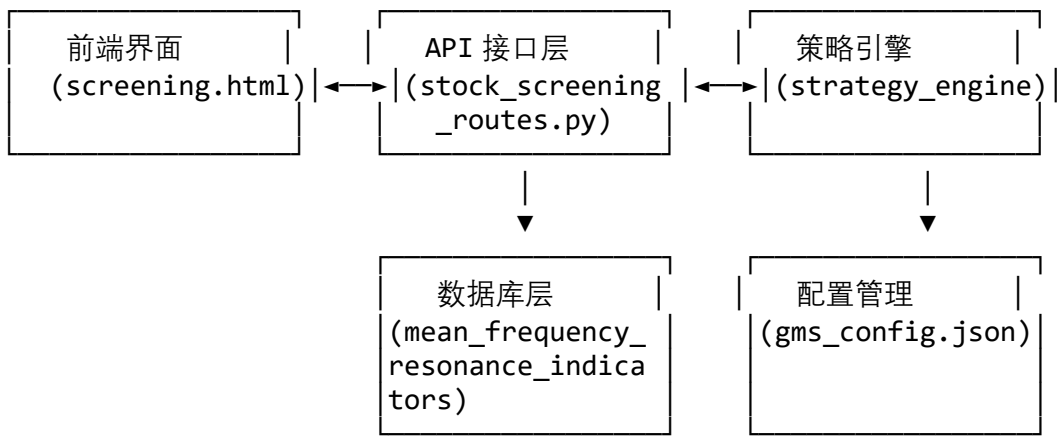


GMS 策略设计文档

1. 系统架构设计

1.1 整体架构



1.2 模块组织

```
backend_core/strategies/gms/
├── __init__.py           # 模块导出
├── models.py             # 数据模型定义
├── config.py             # 配置管理器
├── interfaces.py         # 接口定义
├── data_loader.py        # 数据加载器
├── indicators_calculator.py # 指标计算器
├── signal_detector.py    # 信号检测器
├── strategy_engine.py    # 策略引擎
├── frontend_interface.py # 前端接口
└── gms_config.json       # 默认配置文件
```

2. 核心组件设计

2.1 数据模型设计

2.1.1 GMSIndicators 数据结构

```
@dataclass
class GMSIndicators:
    # 基础数据
    code: str           # 股票代码
    date: str           # 日期
    market_type: str    # 市场类型 CN/HK
```

```

# 原始指标 (来自数据表)
delta: float          # 宏观位移  $\Delta = d_{20} - d_1$ 
d: float              # 20 日均价
ratio_d20: Optional[float] # 偏离率  $\Delta/d_{20}$ 
ratio_d1: Optional[float]  # 突变率  $\Delta/d_1$ 
instant_deviation: float   #  $d_{20} - d$  (价格 vs 均线)
rising_days: int          # Z 上涨天数
falling_days: int         # F 下跌天数
avg_volume_20d: float     # m 20 日平均成交量
current_volume: float     #  $m_{20}$  当日成交量

# 衍生计算指标
ratio_d: Optional[float]  #  $\Delta/d$  相对位移
volume_ratio: Optional[float] #  $m_{20}/m$  量比
fz_ratio: Optional[float]  # F/Z 数方比

# 双模块评分
score_accumulation: float # 均值收敛态总分 0-100
score_momentum: float     # 动量溢出态总分 0-100
score_total: float        # 综合总分

# 执行等级
accumulation_grade: str    # S/A/空
momentum_grade: str       # 全速切入/分批买入/空

# 各维度得分
score_acc_fz: float        # 时间耗散得分
score_acc_balance: float   # 引力粘合得分
score_acc_volume: float    # 成交量缩得分
score_mom_ratio_d1: float  # 盈亏反转得分
score_mom_deviation: float # 推力支撑得分
score_mom_volume: float    # 攻击强度得分

# 买卖信号
left_buy_signal: bool      # 左侧买点
right_buy_signal: bool     # 右侧买点
sell_signal: bool          # 卖点

```

2.1.2 GMSSignal 信号结构

```

@dataclass
class GMSSignal:
    symbol: str          # 股票代码
    date: str            # 信号日期

```

```

signal_type: GMSSignalType    # 信号类型
price: float                  # 信号价格
strength: float                # 信号强度 0-1
reason: str                    # 信号原因
indicators: Optional[GMSIndicators] # 关联指标
conditions_met: Dict[str, bool] # 满足的条件

```

2.2 配置管理设计

2.2.1 配置结构

```

class GMSConfigManager:
    def get_default_config(self) -> Dict:
        return {
            "observation_period": 20,
            "ratio_indicators": {
                "use_ratio_d": True,
                "use_ratio_d_for_exit": False
            },
            "left_buy": {
                "ratio_d20_abs_max": 0.015, # 1.5%
                "volume_ratio_max": 0.8
            },
            "right_buy": {
                "volume_ratio_min": 1.5
            },
            "scoring": {
                # 基础阈值
                "accumulation_fz_min": 1.5,
                "balance_ratio_max": 0.01,
                "momentum_volume_ratio_min": 1.5,
                "watch_threshold": 60,
                "alert_threshold": 90,

                # 阶梯配置
                "accumulation_fz_tiers": [2.5, 1.5],
                "balance_ratio_d_tiers": [0.01, 0.015],
                "volume_ratio_shrink_tiers": [0.6, 0.8],
                "ratio_d1_tiers": [0.001, 0.03],
                "volume_ratio_attack_tiers": [2.0, 1.5],

                # 等级阈值
                "accumulation_s_threshold": 85,
                "accumulation_a_threshold": 70,
                "momentum_full_threshold": 90,
                "momentum_batch_threshold": 80,

                # 评分权重
                "weight_acc_fz": 30,

```

```

        "weight_acc_balance": 40,
        "weight_acc_volume": 30,
        "weight_mom_ratio_d1": 40,
        "weight_mom_deviation": 30,
        "weight_mom_volume": 30
    },
    "exit": {
        "trend_break_days": 3,
        "overbought_ratio": 0.15
    }
}

```

2.3 数据加载器设计

2.3.1 GMSDataLoader 接口

```

class GMSDataLoader:
    def load_indicators(
        self,
        codes: List[str],
        date: str,
        market_type: str = "CN",
        use_latest_per_stock: bool = False
    ) -> List[Dict[str, Any]]:
        """加载单日指标数据"""

    def load_indicators_multi_day(
        self,
        codes: List[str],
        end_date: str,
        market_type: str = "CN",
        days: int = 3
    ) -> List[Dict[str, Any]]:
        """加载多日指标数据"""

```

2.3.2 数据处理流程

原始数据表字段 → 标准化处理 → 衍生指标计算 → 返回字典格式

```

mean_frequency_resonance_indicators:
├── macro_displacement_delta → delta
├── ma20_d → d
├── ratio_d20 → ratio_d20
├── ratio_d1 → ratio_d1
├── instant_deviation → instant_deviation
├── rising_days_z → rising_days
├── falling_days_f → falling_days
├── mavol20_m → avg_volume_20d
├── efficiency_m20_minus_m → 计算 current_volume
└── bias → ratio_d

```

衍生计算:

- $\text{volume_ratio} = \text{current_volume} / \text{avg_volume_20d}$
- $\text{fz_ratio} = \text{falling_days} / \text{rising_days}$ (when $\text{rising_days} > 0$)
- $\text{abs_ratio_d} = \text{abs}(\text{delta} / d)$

2.4 指标计算器设计

2.4.1 双模块阶梯式评分算法

均值收敛态评分算法:

```
def calculate_accumulation_score(self, indicators: GMSIndicators) -> float:
    score = 0.0

    # 1. 时间耗散  $F/Z$  (权重30)
    if fz_ratio >= 2.5:
        score += 30
    elif fz_ratio >= 1.5:
        score += 20 # 2/3 * 30

    # 2. 引力粘合  $|\Delta/d|$  (权重40)
    if abs_ratio_d <= 0.01:
        score += 40
    elif abs_ratio_d <= 0.015:
        score += 20 # 1/2 * 40

    # 3. 成交量缩  $m_{20}/m$  (权重30)
    if volume_ratio <= 0.6:
        score += 30
    elif volume_ratio <= 0.8:
        score += 15 # 1/2 * 30

    return score
```

动量溢出态评分算法:

```
def calculate_momentum_score(self, indicators: GMSIndicators) -> float:
    score = 0.0

    # 1. 盈亏反转  $\Delta/d_1$  (权重40)
    if 0 < ratio_d1 <= 0.001:
        score += 20 # 1/2 * 40
    elif 0.001 < ratio_d1 <= 0.03:
        score += 40 # 满分
    elif ratio_d1 > 0.03:
        score += 0 # 追高

    # 2. 推力支撑  $d_{20}-d$  (权重30)
```

```

if instant_deviation <= 0:
    score -= 10 # 固定负分
elif is_stable_3_days:
    score += 30 # 站稳3日
else:
    score += 15 # 仅当日

# 3. 攻击强度  $m_{20}/m$  (权重30)
if volume_ratio >= 2.0:
    score += 30
elif volume_ratio >= 1.5:
    score += 20 #  $2/3 * 30$ 

return score

```

2.4.2 等级判定逻辑

```

def determine_grades(self, indicators: GMSIndicators):
    # 均值收敛态等级
    if indicators.score_accumulation >= 85:
        indicators.accumulation_grade = "S"
    elif indicators.score_accumulation >= 70:
        indicators.accumulation_grade = "A"

    # 动量溢出态等级
    if indicators.score_momentum >= 90:
        indicators.momentum_grade = "全速切入"
    elif indicators.score_momentum >= 80:
        indicators.momentum_grade = "分批买入"

```

2.5 信号检测器设计

2.5.1 左侧买点检测算法

```

def detect_left_buy(self, indicators: GMSIndicators) -> bool:
    # 优先使用等级判断
    if indicators.accumulation_grade in ("S", "A"):
        return True

    # 传统条件判断
    conditions = [
        indicators.falling_days > indicators.rising_days, #  $F > Z$ 
        indicators.delta < 0, #  $d_{20} < d_1$ 
        abs(indicators.ratio_d20 or 0) < 0.015, # 极度粘合
        (indicators.volume_ratio or 1.0) < 0.8 # 地量
    ]

    return all(conditions)

```

2.5.2 右侧买点检测算法

```
def detect_right_buy(self, indicators: GMSIndicators) -> bool:
    # 优先使用等级判断
    if indicators.momentum_grade in ("全速切入", "分批买入"):
        return True

    # 传统条件判断
    conditions = [
        indicators.instant_deviation > 0, #  $d_{20} > d$ 
        indicators.delta > 0, #  $\Delta > 0$ 
        (indicators.volume_ratio or 0) >= 1.5 # 位移放量
    ]

    return all(conditions)
```

2.6 策略引擎设计

2.6.1 选股流程

```
def screen(self, codes: List[str], date: str, market: str) -> List[Dict]:
    results = []

    # 1. 数据加载
    rows = self.data_loader.load_indicators(codes, date, market, use_latest_per_stock=True)

    # 2. 多日数据加载 (用于站稳3日判断)
    if self.stable_days > 1:
        multi_rows = self.data_loader.load_indicators_multi_day(codes,
            date, market, self.stable_days)
        dev_series_by_code = self._build_deviation_series(multi_rows)

    # 3. 逐股票处理
    for row in rows:
        # 3.1 计算指标
        indicators = self.calculator.calculate(row, dev_series_by_code.get(row['code']))

        # 3.2 信号检测
        left_buy = self.detector.detect_left_buy(indicators)
        right_buy = self.detector.detect_right_buy(indicators)
        sell = self.detector.detect_sell(indicators)

        # 3.3 构建结果
        result = self._build_result(indicators, left_buy, right_buy, sell)
        results.append(result)
```

11)

4. 排序返回

```
results.sort(key=lambda x: x['score_total'], reverse=True)
return results
```

3. API 接口设计

3.1 选股接口设计

3.1.1 接口定义

```
@router.get("/gms-strategy")
async def get_gms_strategy(
    # 基础参数
    date: str = Query(None, description="目标日期 YYYY-MM-DD"),
    limit: int = Query(None, ge=1, description="最大返回数量"),
    min_score: float = Query(0, ge=0, le=100, description="最低总分阈值"),
    scope: str = Query("all", description="股票范围: all/cn/hk/watchlist"),
    code: Optional[str] = Query(None, description="单个股票代码"),

    # 策略参数 (可覆盖配置文件)
    accumulation_fz_min: Optional[float] = Query(None),
    balance_ratio_max: Optional[float] = Query(None),
    volume_ratio_min: Optional[float] = Query(None),
    # ... 其他参数

    # 认证和数据库
    token: Optional[str] = Depends(oauth2_scheme_optional),
    db: Session = Depends(get_db)
):
```

3.1.2 参数处理流程

1. 日期处理

```
if not date:
    # 从历史行情表获取最新日期
    date = get_latest_trading_date(db)
```

2. 股票池确定

```
if code:
    stock_pool = [code]
elif scope == "watchlist":
    stock_pool = get_user_watchlist(token, db)
elif scope == "cn":
    market = "cn"
elif scope == "hk":
    market = "hk"
```



```

else:
    market = "all"

# 3. 配置覆盖
config = GMSConfigManager().get_config()
if accumulation_fz_min is not None:
    config["scoring"]["accumulation_fz_min"] = accumulation_fz_min
# ... 其他参数覆盖

# 4. 执行选股
gms_interface = GMSFrontendInterface(db, config)
results = gms_interface.get_selection_results(date, stock_pool, market)

```

3.1.3 响应格式设计

```

{
    "success": true,
    "data": [
        {
            # 基础信息
            "symbol": "000001",
            "code": "000001",
            "name": "平安银行",

            # 评分信息
            "score_total": 85.5,
            "score_accumulation": 85.5,
            "score_momentum": 45.0,
            "accumulation_grade": "S",
            "momentum_grade": "",
            "signal_strength": 0.855,

            # 信号信息
            "buy_type": "左侧",
            "left_buy_signal": true,
            "right_buy_signal": false,
            "sell_signal": false,

            # 价格信息
            "current_price": 12.34,
            "current_change_percent": 1.23,

            # 核心指标
            "delta": 0.15,
            "d": 12.20,
            "ratio_d20": 0.008,
            "ratio_d1": -0.002,
            "ratio_d": 0.012,
            "fz_ratio": 2.1,

```

```

        "volume_ratio": 0.75,
        "instant_deviation": 0.14,
        "rising_days": 6,
        "falling_days": 12,

        # 详细评分
        "score_detail": {
            "score_acc_fz": 20.0,
            "score_acc_balance": 40.0,
            "score_acc_volume": 25.5,
            "score_mom_ratio_d1": 0.0,
            "score_mom_deviation": 15.0,
            "score_mom_volume": 0.0,
            "acc_fz_judge": "达标(2/3)",
            "acc_balance_judge": "达标(满分)",
            "acc_volume_judge": "达标(1/2)",
            "mom_ratio_d1_judge": "未达标( $\leq 0$ )",
            "mom_deviation_judge": "达标(仅当日)",
            "mom_volume_judge": "未达标"
        }
    },
    "total": 1,
    "search_date": "2026-01-16",
    "strategy_name": "GMS 均值引力动量策略",
    "parameters": {
        "limit": 100,
        "min_score": 0,
        "scope": "cn"
    },
    "message": "所选日期无指标数据, 已使用最新可用日期 2026-01-15"
}

```

3.2 错误处理设计

3.2.1 异常类型定义

```
class GMSException(Exception):
```

```
    """GMS 策略基础异常"""
```

```
    pass
```

```
class DataInsufficientException(GMSException):
```

```
    """数据不足异常"""
```

```
    pass
```

```
class CalculationException(GMSException):
```

```
    """计算异常"""
```

```
    pass
```

3.2.2 错误响应格式

```
{
  "success": false,
  "message": "GMS 策略选股失败：数据不足",
  "error_code": "DATA_INSUFFICIENT",
  "data": []
}
```

4. 前端界面设计

4.1 页面结构设计

4.1.1 策略选项卡

```
<button class="strategy-tab" data-strategy="gms" id="gmsTab">
  GMS 均值引力动量
</button>
```

4.1.2 参数配置区域

```
<div class="strategy-params-card" id="gmsStrategyParamsCard">
  <h3>策略参数</h3>

  <!-- 数据来源选择 -->
  <div class="params-grid">
    <input type="radio" name="gmsScope" value="cn" checked>全部 A 股
    <input type="radio" name="gmsScope" value="hk">全部港股
    <input type="radio" name="gmsScope" value="watchlist">我的自选
  </div>

  <!-- 基础参数 -->
  <div class="params-grid gms-params-grid">
    <div class="parameter-group">
      <label for="gms-start_date">策略起始交易日期</label>
      <input type="date" id="gms-start_date" class="param-input">
    </div>
    <!-- 更多参数... -->
  </div>

  <!-- 评分权重 -->
  <div class="params-grid gms-params-grid">
    <h4>评分权重（每模块合计建议 100）</h4>
    <div class="parameter-group">
      <label for="gms-weight_acc_fz">均值收敛态 时间耗散 F/Z 权重</label>
      <input type="number" id="gms-weight_acc_fz" value="30">
    </div>
    <!-- 更多权重参数... -->
  </div>
</div>
```

```
    </div>
</div>
```

4.1.3 结果展示表格

```
<table class="results-table">
  <thead>
    <tr>
      <th>股票代码</th>
      <th>股票名称</th>
      <th style="display:none;">总分</th>
      <th>信号强度</th>
      <th>买点类型</th>
      <th>当前价格</th>
      <th> $\Delta$  (20 日位移)</th>
      <th>F (下跌天)</th>
      <th>Z (上涨天)</th>
      <th>d (20 日均价)</th>
      <th> $\Delta/d$  (位移/均价)</th>
      <th> $\Delta/d_{20}$ </th>
      <th> $\Delta/d_1$ </th>
      <th>F/Z</th>
      <th>当前涨跌幅</th>
      <th>操作</th>
    </tr>
  </thead>
  <tbody id="resultsTableBody-gms">
    <!-- 动态填充结果 -->
  </tbody>
</table>
```

4.2 交互逻辑设计

4.2.1 参数加载和保存

// 加载 GMS 参数

```
loadGmsParams() {
  const params = this.getStoredGmsParams();
  Object.keys(params).forEach(key => {
    const element = document.getElementById(`gms-${key}`);
    if (element) {
      element.value = params[key];
    }
  });
}
```

// 保存 GMS 参数

```
saveGmsParams() {
  const params = {};
```

```

        document.querySelectorAll('[id^="gms-"]').forEach(element => {
            const key = element.id.replace('gms-', '');
            params[key] = element.value;
        });
        localStorage.setItem('gmsParams', JSON.stringify(params));
    }

```

4.2.2 选股请求处理

```

async refreshGmsStrategy() {
    const loadingIndicator = document.getElementById('loadingIndicator-gms');
    const errorMessage = document.getElementById('errorMessage-gms');

    try {
        loadingIndicator.style.display = 'block';
        errorMessage.style.display = 'none';

        // 构建请求参数
        const params = this.buildGmsRequestParams();

        // 发送请求
        const response = await fetch(`${this.API_BASE_URL}/api/screenin
g/gms-strategy?${params}`);
        const data = await response.json();

        if (data.success) {
            this.displayGmsResults(data.data);
            this.updateResultsCount('gms', data.total);
            this.updateSearchDate('gms', data.search_date);
        } else {
            throw new Error(data.message || 'GMS 策略选股失败');
        }
    } catch (error) {
        this.showError('gms', error.message);
    } finally {
        loadingIndicator.style.display = 'none';
    }
}

```

4.2.3 结果展示处理

```

displayGmsResults(results) {
    const tbody = document.getElementById('resultsTableBody-gms');

    if (!results || results.length === 0) {
        tbody.innerHTML = '<tr><td colspan="16" class="empty-state">暂
无符合条件的股票</td></tr>';
        return;
    }
}

```

```
tbody.innerHTML = results.map(stock => `
    <tr>
        <td><a href="stock.html?code=${stock.code}" class="stock-link">${stock.code}</a></td>
        <td>${stock.name}</td>
        <td style="display:none;">${stock.score_total}</td>
        <td><span class="signal-strength" data-strength="${stock.signal_strength}">${(stock.signal_strength * 100).toFixed(1)}%</span></td>
        <td><span class="buy-type ${stock.buy_type === '左侧' ? 'left-buy' : stock.buy_type === '右侧' ? 'right-buy' : ''}">${stock.buy_type || '-'}</span></td>
        <td>${stock.current_price?.toFixed(2) || '-'}</td>
        <td>${stock.delta?.toFixed(4) || '-'}</td>
        <td>${stock.falling_days || '-'}</td>
        <td>${stock.rising_days || '-'}</td>
        <td>${stock.d_ma20?.toFixed(2) || '-'}</td>
        <td>${stock.ratio_relative?.toFixed(4) || '-'}</td>
        <td>${stock.ratio_d20?.toFixed(4) || '-'}</td>
        <td>${stock.ratio_d1?.toFixed(4) || '-'}</td>
        <td>${stock.fz_ratio?.toFixed(2) || '-'}</td>
        <td class="change-percent ${stock.current_change_percent >= 0 ? 'positive' : 'negative'}">
            ${stock.current_change_percent?.toFixed(2) || 0}%
        </td>
        <td>
            <button class="action-btn" onclick="ScreeningPage.addToWatchlist('${stock.code}')">加自选</button>
            <button class="action-btn" onclick="ScreeningPage.showGmsDetail('${stock.code}')">详情</button>
        </td>
    </tr>
` ).join('');
}
```

5. 数据库设计

5.1 依赖表结构

5.1.1 mean_frequency_resonance_indicators 表

```
CREATE TABLE mean_frequency_resonance_indicators (
    id SERIAL PRIMARY KEY,
    code VARCHAR(20) NOT NULL,
    date VARCHAR(20) NOT NULL,
    market_type VARCHAR(10) NOT NULL,
```

-- 核心指标

macro_displacement_delta REAL, -- Δ 宏观位移

```

    amplitude REAL,           -- 振幅
    ratio_d20 REAL,           --  $\Delta/d_{20}$  偏离率
    ratio_d1 REAL,            --  $\Delta/d_1$  突变率
    instant_deviation REAL,    --  $d_{20}-d$  瞬时偏离
    rising_days_z INTEGER,     -- Z 上涨天数
    falling_days_f INTEGER,    -- F 下跌天数
    efficiency_m20_minus_m REAL, -- 效率指标
    ma20_d REAL,               -- d 20 日均价
    mavol20_m REAL,            -- m 20 日平均成交量
    bias REAL,                 --  $\Delta/d$  乖离率

    -- 价格锚点
    d1 REAL,                   --  $d_1$  起点价格
    d1_date VARCHAR(20),       --  $d_1$  对应日期
    d20 REAL,                   --  $d_{20}$  终点价格
    d20_date VARCHAR(20),      --  $d_{20}$  对应日期

    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    UNIQUE(code, date, market_type)
);

-- 索引优化
CREATE INDEX idx_mfri_code_date ON mean_frequency_resonance_indicators
(code, date);
CREATE INDEX idx_mfri_market_date ON mean_frequency_resonance_indicators
(market_type, date);

```

5.2 查询优化设计

5.2.1 单日数据查询

```

-- 基础查询
SELECT code, date, market_type,
    macro_displacement_delta, ma20_d, ratio_d20, ratio_d1,
    instant_deviation, rising_days_z, falling_days_f,
    mavol20_m, efficiency_m20_minus_m, bias,
    d1, d1_date, d20, d20_date
FROM mean_frequency_resonance_indicators
WHERE code IN (:codes)
    AND date = :date
    AND market_type = :market_type;

-- 最新可用日查询
SELECT DISTINCT ON (code) code, date, market_type, ...
FROM mean_frequency_resonance_indicators
WHERE code IN (:codes)
    AND date <= :date

```

```
    AND market_type = :market_type
ORDER BY code, date DESC;
```

5.2.2 多日数据查询

-- 最近N日数据 (用于站稳3日判断)

```
WITH ranked_data AS (
    SELECT *,
           ROW_NUMBER() OVER (PARTITION BY code ORDER BY date DESC) as
rn
    FROM mean_frequency_resonance_indicators
    WHERE code IN (:codes)
          AND date <= :end_date
          AND market_type = :market_type
)
SELECT * FROM ranked_data WHERE rn <= :days
ORDER BY code, date;
```

6. 性能优化设计

6.1 数据加载优化

6.1.1 批量查询策略

避免逐个股票查询, 使用批量查询

```
def load_indicators_batch(self, codes: List[str], date: str, market_type: str):
    # 单次查询获取所有股票数据
    query = self.db.query(MeanFrequencyResonanceIndicators).filter(
        MeanFrequencyResonanceIndicators.code.in_(codes),
        MeanFrequencyResonanceIndicators.date == date,
        MeanFrequencyResonanceIndicators.market_type == market_type
    )
    return query.all()
```

6.1.2 缓存策略

配置缓存 (可选)

@lru_cache(maxsize=1000)

```
def get_cached_indicators(self, code: str, date: str, market_type: str):
    """缓存单股票指标数据"""
    pass
```

6.2 计算优化

6.2.1 向量化计算

使用NumPy进行批量计算

```
import numpy as np
```

```
def calculate_batch_scores(self, indicators_list: List[GMSIndicators]):
    """批量计算评分, 提升性能"""
```



```

# 提取数组
fz_ratios = np.array([ind.fz_ratio or 0 for ind in indicators_list])
abs_ratio_ds = np.array([abs(ind.delta / ind.d) if ind.d > 0 else 0
for ind in indicators_list])

# 向量化计算
fz_scores = np.where(fz_ratios >= 2.5, 30,
                     np.where(fz_ratios >= 1.5, 20, 0))

return fz_scores

```

6.3 API 响应优化

6.3.1 异步处理

```

import asyncio

async def get_gms_strategy(...):
    # 使用线程池执行CPU密集型任务
    loop = asyncio.get_event_loop()

    def _run_screening():
        return gms_interface.get_selection_results(date, stock_pool, ma
rket)

    results = await asyncio.wait_for(
        loop.run_in_executor(None, _run_screening),
        timeout=30 # 30 秒超时
    )

    return results

```

6.3.2 分页处理

大数据量时支持分页

```

def screen_with_pagination(self, codes: List[str], page: int = 1, page_
size: int = 100):
    start_idx = (page - 1) * page_size
    end_idx = start_idx + page_size

    # 分批处理
    batch_codes = codes[start_idx:end_idx]
    return self.screen(batch_codes, ...)

```

7. 测试设计

7.1 单元测试设计

7.1.1 指标计算测试

```
class TestGMSIndicatorsCalculator:
    def test_accumulation_score_calculation(self):
        """测试均值收敛态评分计算"""
        calculator = GMSIndicatorsCalculator()

        # 测试满分情况
        indicators = GMSIndicators(
            fz_ratio=3.0, # >= 2.5
            abs_ratio_d=0.005, # <= 0.01
            volume_ratio=0.5 # <= 0.6
        )

        score = calculator._score_accumulation_fz(indicators.fz_ratio)
        assert score == 30

    def test_momentum_score_calculation(self):
        """测试动量溢出态评分计算"""
        # 测试各种边界条件
        pass
```

7.1.2 信号检测测试

```
class TestGMSSignalDetector:
    def test_left_buy_detection(self):
        """测试左侧买点检测"""
        detector = GMSSignalDetector()

        # 测试满足条件的情况
        indicators = GMSIndicators(
            accumulation_grade="S",
            falling_days=15,
            rising_days=5,
            delta=-0.1,
            ratio_d20=0.01,
            volume_ratio=0.7
        )

        assert detector.detect_left_buy(indicators) == True

    def test_right_buy_detection(self):
        """测试右侧买点检测"""
        # 测试各种情况
        pass
```

7.2 集成测试设计

7.2.1 端到端测试

```
class TestGMSIntegration:
    def test_full_screening_process(self):
        """测试完整选股流程"""
        # 1. 准备测试数据
        test_codes = ["000001", "000002"]
        test_date = "2026-01-16"

        # 2. 执行选股
        engine = GMSStrategyEngine(data_loader, config)
        results = engine.screen(test_codes, test_date, "CN")

        # 3. 验证结果
        assert isinstance(results, list)
        for result in results:
            assert "symbol" in result
            assert "score_total" in result
            assert result["score_total"] >= 0
```

7.2.2 API 测试

```
class TestGMSAPI:
    def test_gms_strategy_api(self):
        """测试GMS策略API"""
        response = client.get("/api/screening/gms-strategy?scope=cn&limit=10")

        assert response.status_code == 200
        data = response.json()
        assert data["success"] == True
        assert "data" in data
        assert len(data["data"]) <= 10
```

7.3 性能测试设计

7.3.1 负载测试

```
class TestGMSPerformance:
    def test_large_dataset_performance(self):
        """测试大数据集性能"""
        import time

        # 测试5000只股票的处理时间
        large_codes = [f"{i:06d}" for i in range(1, 5001)]

        start_time = time.time()
        results = engine.screen(large_codes, "2026-01-16", "CN")
        end_time = time.time()
```

```

        processing_time = end_time - start_time
        assert processing_time < 30 # 30 秒内完成

    def test_concurrent_requests(self):
        """测试并发请求性能"""
        # 模拟多个并发请求
        pass

```

8. 部署和运维设计

8.1 配置管理

8.1.1 环境配置

生产环境配置

```

PRODUCTION_CONFIG = {
    "scoring": {
        "watch_threshold": 70, # 生产环境提高阈值
        "alert_threshold": 95
    },
    "performance": {
        "max_concurrent_requests": 10,
        "request_timeout": 30
    }
}

```

开发环境配置

```

DEVELOPMENT_CONFIG = {
    "scoring": {
        "watch_threshold": 50, # 开发环境降低阈值便于测试
        "alert_threshold": 80
    }
}

```

8.2 监控和日志

8.2.1 日志设计

```

import logging

logger = logging.getLogger('gms_strategy')

# 关键操作日志
logger.info(f"GMS 选股开始: codes={len(codes)}, date={date}, market={market}")
logger.info(f"GMS 选股完成: 找到{len(results)}只股票, 耗时{elapsed_time:.2f}秒")

```

错误日志

```
logger.error(f"GMS 指标计算失败: {error}", exc_info=True)
```

性能日志

```
logger.info(f"GMS 性能指标: 平均每股处理时间{avg_time:.4f}秒")
```

8.2.2 监控指标

关键监控指标

```
MONITORING_METRICS = {  
    "request_count": "GMS 策略请求总数",  
    "success_rate": "GMS 策略成功率",  
    "avg_response_time": "平均响应时间",  
    "error_rate": "错误率",  
    "concurrent_users": "并发用户数"  
}
```

8.3 扩展性设计

8.3.1 水平扩展

支持分布式处理

```
class DistributedGMSEngine:  
    def screen_distributed(self, codes: List[str], workers: int = 4):  
        """分布式选股处理"""  
        from multiprocessing import Pool  
  
        # 将股票代码分组  
        code_chunks = [codes[i::workers] for i in range(workers)]  
  
        # 并行处理  
        with Pool(workers) as pool:  
            results = pool.map(self._screen_chunk, code_chunks)  
  
        # 合并结果  
        return [item for sublist in results for item in sublist]
```

8.3.2 缓存扩展

Redis 缓存支持

```
import redis
```

```
class GMSCacheManager:
```

```
    def __init__(self):
```

```
        self.redis_client = redis.Redis(host='localhost', port=6379, db=0)
```

```
    def get_cached_result(self, cache_key: str):
```

```
        """获取缓存结果"""
```

```
cached = self.redis_client.get(cache_key)
return json.loads(cached) if cached else None

def set_cached_result(self, cache_key: str, result: dict, expire: int = 300):
    """设置缓存结果"""
    self.redis_client.setex(cache_key, expire, json.dumps(result))
```

这个设计文档涵盖了 GMS 策略的完整技术设计，包括系统架构、核心算法、API 接口、前端界面、数据库设计、性能优化、测试策略和部署运维等各个方面。设计遵循了模块化、可扩展、高性能的原则，为 GMS 策略的实现和维护提供了全面的技术指导。